CENG218 – Labwork 8

Definitions

A binary tree is either empty, or it consists of a node called the root together with two binary trees called the left subtree and the right subtree of the root.

Traversal of Binary Trees

One of the most important operations on a binary tree is ***traversal***, moving through all the nodes of the binary tree, visiting each one in turn. As for traversal of other data structures, the action we shall take when we *visit* each node will depend on the application.

If we name the tasks of visiting a node *V*, traversing the left subtree *L*, and traversing the right subtree *R*, then there are six ways to arrange them:

1. *V L R*
2. *L V R*
3. *L R V*
4. *V R L*
5. *R V L*
6. *R L V*

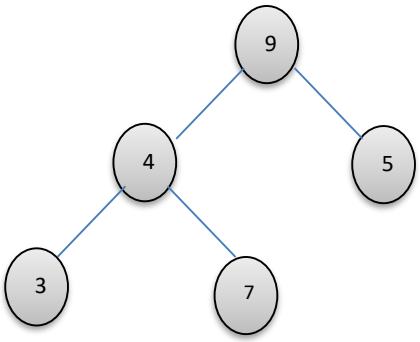
Standard Traversal Orders

By standard convention, these six are reduced to three by considering only the ways in which the left subtree is traversed before the right.

V L R	*L V R*	*L R V*
preorder	*inorder*	*postorder*

Linked Implementation of Binary Trees

Task 1: Build the following binary tree:




```
void main(){

BinaryTree myTree;//default constructor creates binary tree given  figure


.

.

}
```

Task 2

Write necessary recursive function to traverse and print the above binary tree in:

a) **Preorder**
b) **Inorder**
c) **Postorder**
d) **Level Order Traversal**

Task 3

Write function that will return:

a) **number of nodes of a given binary tree.**
b) **number of terminal (leaf) nodes of a given binary tree.**
c) **number of nodes that contain odd number as their data field of a given binary tree.**
d) **The height or depth of a tree is defined to be the maximum level of any node in the tree. (depth=the longest path from root to any leaf node). Write a function that will return height of a given binary tree.**